PROPOSAL · OCTOBER 2026

PDLC harness

Every feature makes the next one smarter

A product development life cycle harness on Claude Code · for product and engineering leaders



Today, every feature starts from zero

FEATURE 1

Decisions are made in a chat thread or a meeting

FEATURE 2

The same questions get asked and answered again

FEATURE 3

The same mistakes get made again

Lost in chat threads, meetings, old docs and people's heads

Claude knows none of it

Every session starts without our standards, services or past decisions.

PMs are outside the loop

They see a ticket, then a demo weeks later; questions wait for meetings.

Everyone works their own way

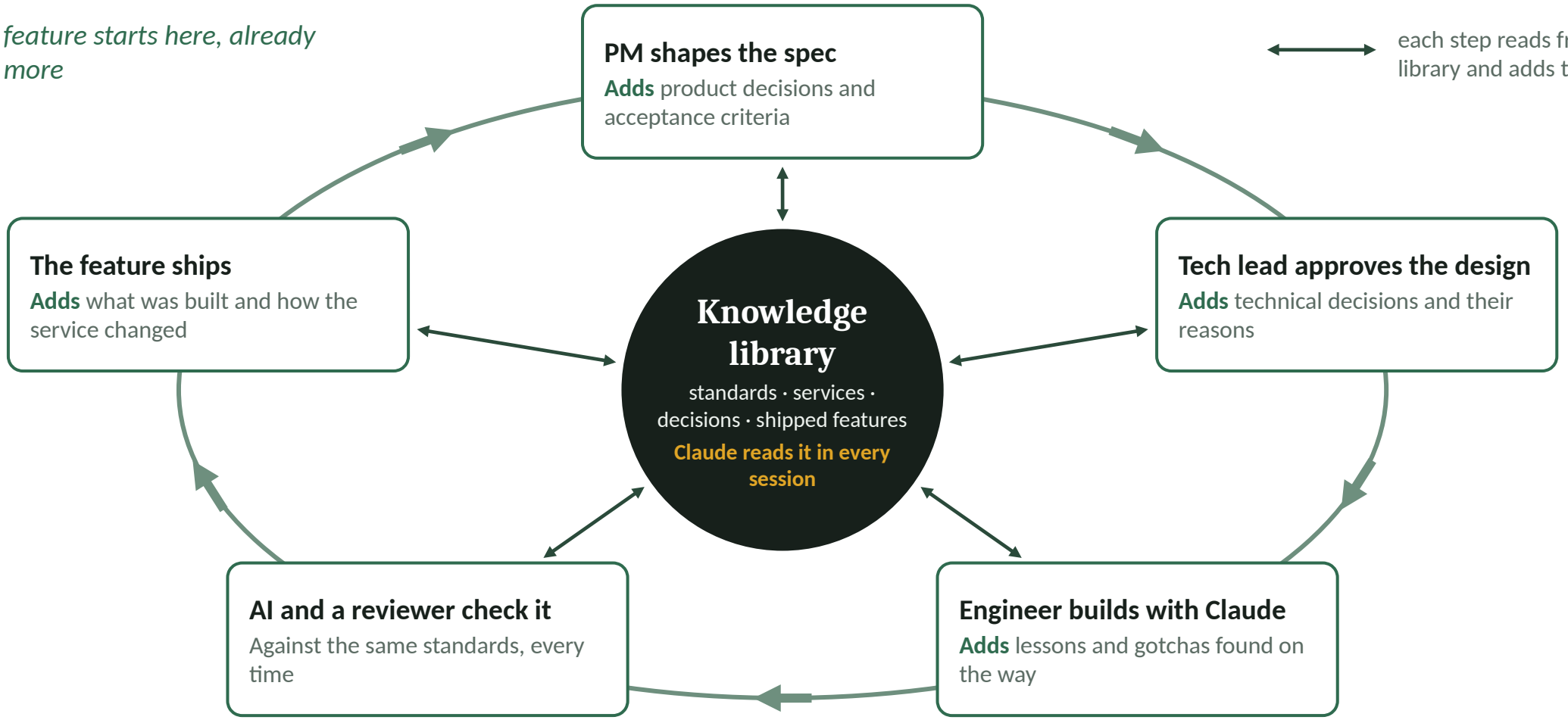
Each team and engineer reads the standards differently.

Today versus with the PDLC harness

	Today	With the PDLC harness
Knowledge	Standards, service details and past decisions live in heads, old docs and chat logs	One knowledge library that every decision and approval adds to; Claude reads it in every session
Standards	Each team reads them differently; reviewers catch issues late and unevenly	Written once by their owners, checked on every PR, dangerous actions blocked
Developer path	Everyone works with AI differently; specs and plans stay in private chats	The same few commands in every repo, ticket to merged code, next step always clear
PM feedback	A ticket, then a demo weeks later; questions wait for meetings	PMs shape and approve the spec, criteria and demo in the PR, within hours
Leadership view	Status, bottlenecks and AI cost are assembled by hand	One dashboard: every story's stage, who it waits on, adoption and cost by team

Every feature makes the next one smarter

The next feature starts here, already knowing more



Decided once, reused by every feature

Who	In their normal work...	...the library keeps
PM	Answers the open questions and approves the spec and acceptance criteria	Product decisions and acceptance criteria
Tech lead	Approves the architecture and the plan	Technical decisions, with the reasons
Engineer	Proposes design choices, and hits the odd gotcha while building	Proposed decisions and lessons learned
Standards owners	Write each standard once and approve every change to it	Standards that apply in every repo
Every merge	An agent drafts page updates after each merged change	Service pages and feature history, kept current

No extra step: it's captured from approvals people already give. Owners approve any new standard or decision.

Claude starts every session knowing us

New session · payments-api *example*

Loaded automatically before the first prompt

SERVICE PAGE

Charges cards and retries failures. Depends on ledger-svc. Gotcha: the card provider times out after 30 seconds.

STANDARDS

Go style, API design, secrets handling, payments security

PAST DECISIONS

Every write carries an idempotency key, decided in PAY-301 with the reasons

YOUR FEATURE

PAY-412 · test plan approved · next: build phase 1 of 3

Code that fits the service

It knows what the service does, what it depends on and where it breaks.

Standards followed from the first draft

The rules for this language and service are in context while it designs and writes.

Settled questions stay settled

Past decisions come with their reasons, so they aren't argued again or quietly undone.

Checked the same way, every time

Every pull request is reviewed against the same standards before a person looks.

From SDLC to PDLC

Today: SDLC

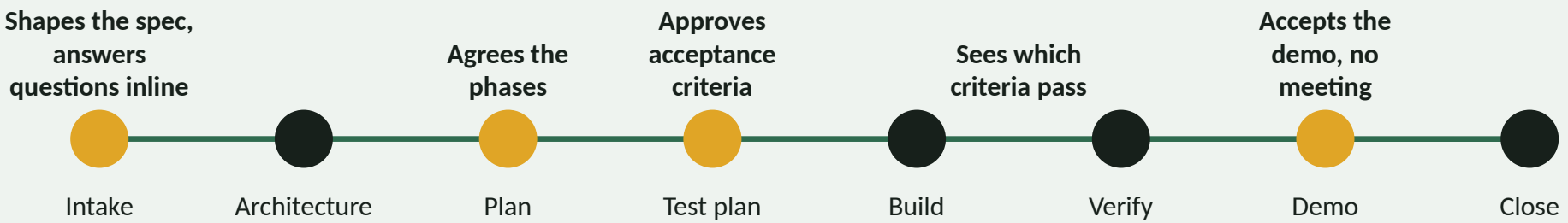
PM at the edges



Weeks of building · questions wait for meetings · feedback arrives late

PDLC harness

PM inside each key step



PM involved

Feedback in hours, not weeks

PMs review where the work happens, as it happens.

Fewer status meetings

Questions are answered inline; status is on the dashboard.

Product decisions kept

Every answer and approval adds to the knowledge library.

Every story's status, without asking anyone

Waiting on me | **All** | By team | Stuck > 2 days | Shipped | Team: all | PM: all | Stage: all | Status: all

Story (opens the PR to approve)	Repo	Stage	Status	Waiting on	For	Phases
PAY-412 Retry failed payments · Test plan	payments-api	4 Test plan	Waiting for approval	PM @asha, QA @lin	2 days	0 / 3
PAY-398 Refund webhooks · Phase 2	payments-api	5 Build	In code review	@tomas (reviewer)	5 hours	1 / 4
SRCH-77 Typo-tolerant search · Spec	search-svc	1 Intake	Changes requested	@kim (developer)	1 day	–
PAY-350 Saved cards · Demo	payments-web	7 Demo	Waiting for approval	PM @asha	4 days	3 / 3



Nobody types a status

Stage, status and who it's waiting on come from the repos and pull requests.



One click to act

Each story opens the pull request that is waiting for approval.



Adoption and cost too

By team, from Claude's built-in analytics, plus a weekly one-page summary.

Built from tools we already use

2

repos: the harness itself and the knowledge library

1

small script is the only new code

3

automated workflows in CI

0

new servers to run

What we configure, not build

Claude Code	One plugin for every engineer, installed and updated centrally
GitHub Spec Kit	The stage commands, with our own templates
GitHub	Pull requests, approval rules, CI and the dashboard
Jira	Tickets in, links back to the work
Claude analytics	Adoption and cost by team

Our first attempt had too many moving parts. This one is sized for a platform team of two.

Pilot first, then grow team by team

1

Foundations

Knowledge library, standards from their owners, first service pages, cost visible

2

Pilot: stages 1–4

One team, one real feature; PMs approve in PRs; dashboard live

3

Pilot: stages 5–8

Build, automated review, demo and close

4

Org rollout

2–3 teams at a time, each adding its service pages

About 6–8 weeks to the pilot · 2 platform engineers · 1 pilot team of 6–8 engineers

How we'll know it works

≥ 90%
of service pages updated within a day of a change

100%
of merged pilot PRs reviewed automatically

≥ 6
pilot engineers using it every week

2
features shipped through all eight stages

< 1 day
median time to a PM's first response

Fewer
status and clarification meetings than at the start

What we need from you

1. **A pilot team and a real feature**
   and that team's count of status and clarification meetings in week 1, as our baseline

2. **GitHub access for PMs and QA**
   with review rights on their team's repos, plus view access for leadership

3. **Our Claude plan confirmed**
   the Enterprise plan reports adoption and cost by team with no extra work

4. **A CI budget per merged PR**
   agreed before the pilot starts

5. **Backing for the pilot rule**
   we expand only once the team uses the approvals instead of skipping them

Next: the platform team settles the open design decisions, then brings a phased plan for review.